

Design and Implementation of a Precise Timeline-Driven Scheduler for FreeRTOS

Academic Report: Project Assignment 2026

Lombardozi Masoumkhani Papantuono Pedron Stella

Politecnico di Torino

Academic Year 2025/2026

Outline

- 1 Architectural Overview and System Initialization
- 2 Memory Management and Static Allocation
- 3 Environment Setup and Emulation
- 4 Kernel Integration and Scheduling Hooks
- 5 Telemetry System and Asynchronous Tracing
- 6 Validation Framework and Automated Testing
- 7 Concluding Remarks

1.1 Design Requirements and Operational Objectives

- Deterministic task scheduling is fundamental in safety-critical embedded systems, where unpredictable execution timing can lead to catastrophic failures.
- **Core Paradigm:** Transition from standard priority-based scheduling to a deterministic *Time-Triggered* (TT) architecture.
- **Timeline Constraints:**
 - Cyclic *Major Frame* and non-overlapping *Minor Frames*.
 - Release jitter bounded strictly to ≤ 1 system tick.
 - Maximum scheduler-induced CPU overhead $\leq 10\%$.
- **Task Models:**
 - **HRT (Hard Real-Time):** Strictly confined to temporal slots.
 - **SRT (Soft Real-Time):** Executed via best-effort during idle time.

1.2 System Configuration and Parsing Phase

Offline Configuration

The system relies on a user-defined JSON configuration file, processed offline by a Python tool.

- **Generation:** Translates high-level directives into an array of statically allocated C structures.
- **Benefits:**
 - Completely eradicates dynamic memory fragmentation.
 - Shifts complexity from runtime to compile-time.

1.3 Data Structures and Internal Representation

- **Core Structure:**
TimelineTaskConfig_t.
- **Common Fields:** Name, Task Function, Type, Task Handle, Static Buffers.
- **HRT Specifics:** ulStart_time, ulEnd_time, Jitter tracking parameters.
- **SRT Specifics:** Embedded ListItem_t node for the static Ready Queue.

```
typedef struct
{
    const char * pcName;           /*< Descriptive name for debug and tracing purposes. */
    TaskFunction_t pvTaskFunction; /*< Pointer to the C function implementing the task payload. */
    TimelineTaskType_t xType;      /*< Task categorization as HRT or SRT. */
    TaskHandle_t xTaskHandle;      /*< FreeRTOS handle, populated dynamically during system initialization. */
    const void * pvTaskParams;     /*< Pointer to task parameters (e.g., workload scaling factors). */

    /* Static Memory Allocation Parameters */
    configSTACK_DEPTH_TYPE xStackDepth; /*< Stack depth specified in words, not bytes. */
    StaticTask_t * pxTaskBuffer;        /*< Pointer to the statically allocated TCB (Task Control Block) memory. */
    StackType_t * pxStackBuffer;        /*< Pointer to the statically allocated stack memory area. */

    /* Temporal Execution Parameters */
    uint32_t ulStart_time; /*< Absolute activation tick relative to the Major Frame boundary. */
    uint32_t ulEnd_time;   /*< Absolute deadline tick relative to the Major Frame boundary. */
    uint32_t ulSubframe_id; /*< Logical sub-frame grouping identifier (Calculated automatically by the kernel). */
    uint32_t ulMaxDelay;    /*< Maximum execution delay recorded during runtime. */
    uint32_t ulMinDelay;    /*< Minimum execution delay recorded during runtime. */

    /* Internal List Management */
    ListItem_t xSRTListItem; /*< Linked list node for enqueueing the task within the SRT timeline ready list. */
} TimelineTaskConfig_t;
```

Figure: TimelineTaskConfig_t definition.

1.4 Application Programming Interfaces (APIs)

- **Information Hiding:** User interaction is restricted to minimal, safe entry points.
- `pxGetTimelineConfig()`: Extracts the auto-generated static task array.
- `vConfigureScheduler()`: Master initialization routine.
 - Orchestrates sorting and rigorous timeline validation.
 - Creates FreeRTOS tasks via `xTaskCreateStatic()`.
 - Suspends all tasks and initializes the static SRT queue before scheduling begins.

1.5 Sorting Algorithms and Execution Priorities

- **Objective:** Sequence the task array to guarantee $\mathcal{O}(1)$ runtime access.
- **Algorithm Choice:** *In-place Stable Insertion Sort*.
- **Why Insertion Sort?**
 - Negligible impact for small N ; strictly $\mathcal{O}(1)$ memory requirement.
 - **Stability:** Crucial for SRT tasks. It maintains their relative user-defined declaration order, establishing implicit execution priorities for the best-effort queue.
 - **Result:** HRT sequenced chronologically; SRT clustered at the tail.

1.6 Temporal Validation and Fault-Tolerance Policies

Static Analysis Phase

Executed by `xTimeline_InitAndValidateTasks()` before the OS starts.

- **Constraints Checked:**
 - Negative time durations.
 - Minor/Major frame boundary overflows.
 - Temporal overlapping of HRT execution slots.
- **Fail-Stop Policy:** Any violation forces an intentional deadlock with diagnostic logs. Tasks are not silently ignored, preventing catastrophic real-world failures.

1.7 Queue Management and Soft Real-Time Execution

- **Architecture:** Utilizes a statically allocated native FreeRTOS `List_t`.
- **No Dynamic Allocation:** `ListItem_t` is directly embedded in the task's configuration matrix.
- **Execution Policy:** Strict First-In, First-Out (FIFO) processing.
- **Cyclic Reset:** At Major Frame boundaries, `vResetSRTQueue()` restores the queue in $\mathcal{O}(1)$ time by merely rewinding internal pointers, avoiding list destruction.

1.8 Workload Simulation and Execution Dynamics

- **Payload Logic:** Compute-intensive multiply-accumulate operations, dictated by a *scaling factor*.
- **Indirection Layer:** Handled via `vGenericTaskWrapper()` to decouple computation from OS signatures.
- **Cooperative Yielding:** Tasks invoke `vTimelineMarkHRTComplete()` to manually yield control early, maximizing the idle time available for SRT execution.

1.9 Context Restoration and Cyclic Reset

- **The Issue:** `vTaskDelete/vTaskCreate` are slow, non-deterministic, and fragment the heap.
- **Custom Primitive:** `vTaskTimelineReset()`.
- **Low-Level Mechanics:**
 - Surgically resets the *Stack Pointer* (SP) to the base.
 - Overwrites the *Program Counter* (PC) with the original entry function.
- **Outcome:** Complete context clearance in rigorous $\mathcal{O}(1)$ time.

2.1 Allocation Strategies: Dynamic vs. Static

- **Design Philosophy:** Complete elimination of dynamic allocation
- **Kernel Configuration:**
 - `configSUPPORT_STATIC_ALLOCATION = 1`
 - `configSUPPORT_DYNAMIC_ALLOCATION = 0`
 - `heap_4.c` stripped from the compilation build.
- **Result:** Eradicates $\mathcal{O}(N)$ latencies and heap exhaustion risks.

2.2 Task-Specific Memory Architecture

Static Provisioning

Memory blocks are allocated at compile time and passed to the kernel.

- **xStackDepth:** Defines specific stack availability per task.
- **pxStackBuffer:** Contiguous memory array physically reserving stack space.
- **pxTaskBuffer:** Memory pre-allocated for the internal Task Control Block (TCB), preventing kernel-level dynamic requests.

3.1 Software Architecture and Repository Layout

- **Modularity:** Strict separation of concerns between OS modifications, application logic, and testing tools.
- **Key Directories:**
 - Demo/: Contains the timeline implementation and payload execution.
 - FreeRTOS/: Houses the patched kernel sources.
 - Scripts in root orchestrate automated builds and validation.

3.2 Cross-Platform Portability and Reproducibility

- **Dockerization:** The build and test environment is fully containerized.
- **Toolchain:** Standardizes the `arm-none-eabi` compiler and QEMU emulator across all host machines.
- **Validation:** Ensures that ≤ 1 tick jitter and $\leq 10\%$ CPU overhead metrics are mathematically reproducible regardless of the developer's underlying OS.

3.3 Virtual Clock Configuration and Determinism

QEMU Instruction Set Simulator (ISS)

Virtual time is strictly coupled to executed instructions to prevent host-load interference.

- **Configuration:** `-icount shift=6, sleep=on`.
- **Clock Downgrade:** Simulates ≈ 15.625 MHz execution.
- **Conservative Validation:** Proving temporal compliance on a degraded clock guarantees absolute safety on the target 25 MHz hardware.

4.1 Execution State Management and Transition

- **Direct Control:** Manual manipulation of task states bypassing native priority lists.
- **Suspension:** `vTaskTimelineSuspend()` removes tasks from active execution outside slots or upon completion.
- **Resumption:** `vTaskTimelineResume()` forcibly wakes tasks exactly at their predefined timeline boundaries.
- **ISR Safe Variants:** Provided for immediate deadline enforcement directly from the hardware timer interrupt.

4.2 Execution Flow Interception and Context Switching

- **xTimelineTickHook:**

- Embedded in SysTick. Tracks global time, detects deadline misses, and evaluates boundary conditions.

- **xTimelineScheduleHook:**

- Embedded in the context switch routine.
- Resolves task selection natively enforcing HRT preemption or delegating to the SRT queue.

4.3 Deterministic Dispatching and Amortized Access

- **Deadline Enforcement:** $\mathcal{O}(1)$ check against the currently running HRT task.
- **Amortized Search Logic:**
 - Avoids $\mathcal{O}(N)$ scans by using `ulHrtTaskIndex`.
 - Since the array is chronologically sorted, the kernel merely checks the index to find the next valid HRT activation time.
- **Slack Time Reclaiming:** Yields seamlessly to Soft Real-Time or system Idle upon early HRT completion.

5.1 Asynchronous Event Logging and Buffer Management

The Print Bottleneck

UART transmission is a blocking operation; placing it inside an ISR destroys Jitter guarantees.

- **Decoupled Design:**

- `vTraceLog()` writes lightweight binary events to a RAM circular buffer in $\mathcal{O}(1)$ time.
- A background consumer task (`vTraceDumperTask`) pulls from RAM and formats text to UART exclusively during idle intervals.

5.2 Offline Analysis and Telemetry Integration

- **Live Monitoring:** Telemetry acts as a direct debug conduit during execution.
- **Data Ingestion:** The `integrated_test.py` script hooks to the serial output.
- **Automated Parsing:** Ingests raw text, identifies key events (Starts, Stops, Misses, Preemptions), and feeds them into the statistical model.

6.1 Pipeline Orchestration and Test Execution

- **End-to-End Automation:** `integrated_test.py` controls compilation, QEMU deployment, logging, and evaluation.
- **Graceful Degradation:** Handles user interrupts while preserving partial log analysis.
- **Regression Testing:** Sequentially executes multiple JSON configurations to stress different workload boundaries.

6.2 Log Parsing and Frame Boundary Detection

- **Data Integrity:** Strips incomplete major frames before statistical evaluation.
- **Boundary Detection:** Correlates explicit log markers with SysTick wrap-around calculations to guarantee exact frame isolation.
- **Information Tallying:** Extracts idle ticks, kernel cycles, and min/max activation delays via regex.
- **Models Abstraction:** Parsing decoupled from metrics generation.

6.3 Metrics Extraction and Jitter Analysis

Jitter Calculation Formula

$$\text{Jitter} = \text{delay}_{\max} - \text{delay}_{\min}$$

Overhead Calculation Formula

$$\text{Cycles per Major Frame} = \left(\frac{\text{configCPU-CLOCK-HZ}}{\text{configTICK-RATE-HZ}} \right) * \text{major's frame ticks}$$

$$\text{Overhead}(\%) = \left(\frac{\text{overhead cycles}}{\text{cycles per Major Frame}} \right) * 100$$

- **Quantitative Proof:** Evaluates max jitter against the ≤ 1 tick hard constraint.
- **Overhead Proof:** Averages consumed scheduler cycles vs total frame cycles to guarantee the 10% threshold limit.

Workload Diagram / CPU Utilitiaion

Task Distribution & Overhead - 20260227_095716_Reduced major frame - tight spacing - mixed HRT/SRT

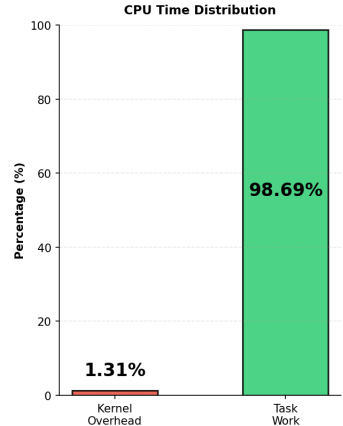
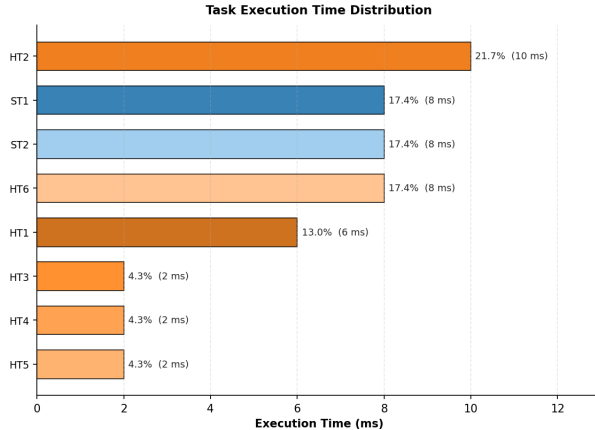


Figure: Detailed view of the specific kernel modification or scheduling trace.

6.4 Visualization Generation and Regression Checking

- **Timeline Plotting:** Visual confirmation of HRT slots (Orange) and SRT best-effort (Blue).
- **Overhead Charting:** Visual verification of kernel consumption vs Task payload.
- **Pass/Fail verdicts:** Automated checks against expected values.

Architectural Diagram / Code Snippet

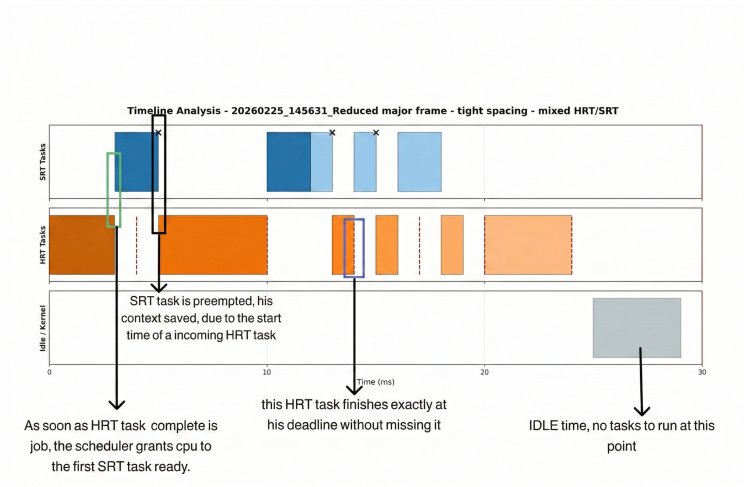


Figure: Detailed view of the specific kernel modification or scheduling trace.

7. Concluding Remarks and Final Assessment

- **Absolute Predictability:** Validation empirically demonstrates strict activation determinism across cyclic repetitions.
- **Requirements Mastered:** Tested configurations confirm the Jitter ≤ 1 tick and the CPU overhead strictly $\leq 10\%$.
- **Architectural Solidity:** The $\mathcal{O}(1)$ static memory approach and custom context restoration have successfully satisfied all the constraints of safety-critical embedded operations.

Thank you for your attention.